

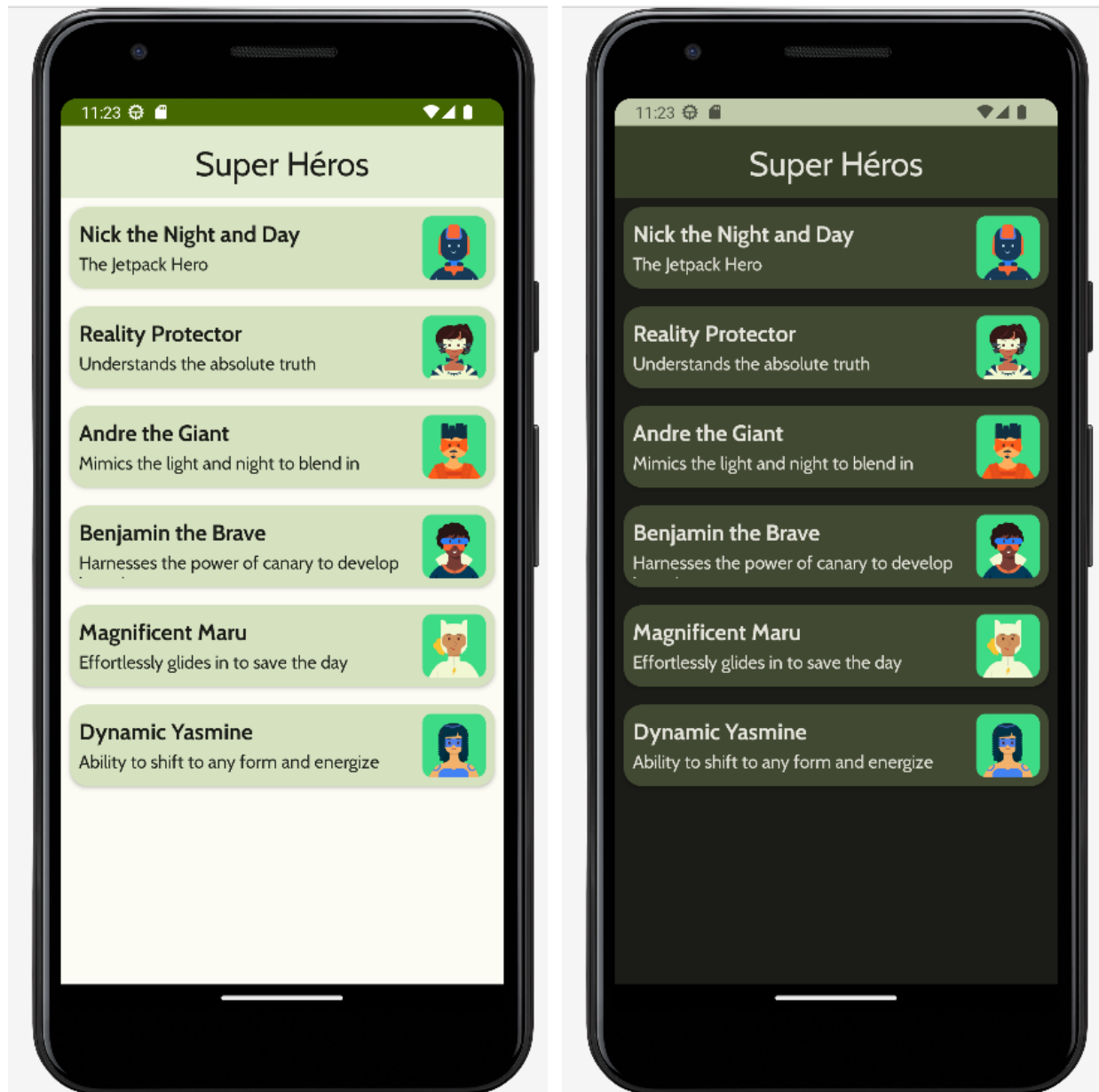
Exercices - Application sur les super-héros

Table des matières

Premiers pas	2
Thématisation Material	3
Couleur	3
Forme	4
Typographie	4
Afficher la liste	5
Barre d'application supérieure	8
Références	8

Dans ces exercices guidés, vous développerez les concepts que vous avez étudiés dans ce parcours en créant une application de **super-héros**. Cette application se concentre sur la conception des composants nécessaires à la création d'une liste déroulante et d'une interface utilisateur soignée à l'aide des principes *Material Design* que vous avez appris dans le chapitre *Material Design*.

Voici ce à quoi la version finale de l'application en thème clair et en thème sombre :



Premiers pas

Dans cette tâche, vous configurerez le projet et créerez les données factices correspondant aux

super-héros.

1. Créer un nouveau projet avec le modèle *Empty Activity*.
2. Télécharger les éléments de l'application (images de super-héros et logo de l'application) sur Omnivox/LEA ([ImagesHeros.zip](#)).
3. Télécharger les fichiers de police Cabin gras et Cabin standard sur <https://fonts.google.com>.
4. Créer une classe de données pour stocker les données de chaque super-héros. N'hésitez pas à créer un package nommé `modele` pour la classe de données principale afin d'organiser votre code. Chaque élément de la liste des super-héros affiche trois informations uniques :
 - Le nom;
 - La description;
 - L'image.
5. Dans un package `data`, créez un autre fichier contenant les données. Voici un exemple d'ensemble de données pour vous inspirer.

```
1 class DataSource {  
2     fun loadData(): List<Heros>{  
3         return listOf<Heros>(  
4             Heros("Nick the Night and Day", "The Jetpack Hero", R.drawable  
               .android_superhero1),  
5             Heros("Reality Protector", "Understands the absolute truth", R  
               .drawable.android_superhero2),  
6             Heros("Andre the Giant", "Mimics the light and night to blend in",  
               R.drawable.android_superhero3),  
7             ...  
8             ...  
9         )  
10    }  
11 }
```

Thématisation Material

Dans cette section, vous ajouterez la palette de couleurs, la typographie et les formes de l'application afin d'améliorer son apparence.

Couleur

Voici une recommandation pour le jeu de couleurs. N'hésitez pas à en choisir un autre.

`ui.theme.Color.kt`

```

1 //Light Theme
2 val md_theme_light_background = Color(0xFFFFDFCF4)
3 val md_theme_light_surface = Color(0xFFE0EACE)
4 val md_theme_light_secondary = Color(0xFF596148)
5 val md_theme_light_onSurface = Color(0xFF1B1C18)
6 val md_theme_light_primary = Color(0xFF466800)
7 val md_theme_light_onPrimary = Color(0xFF223600)
8
9 // Dark Theme
10 val md_theme_dark_background = Color(0xFF1B1C18)
11 val md_theme_dark_surface = Color(0xFF373F29)
12 val md_theme_dark_secondary = Color(0xFFDDE6C6)
13 val md_theme_dark_onSurface = Color(0xFFE4E3DB)
14 val md_theme_dark_primary = Color(0xFFC1CAAB)
15 val md_theme_dark_onPrimary = Color(0xFFDDE6C6)

```

Forme

`ui.theme.Shape.kt`

```

1 import androidx.compose.foundation.shape.RoundedCornerShape
2 import androidx.compose.material3.Shapes
3 import androidx.compose.ui.unit.dp
4
5 val Shapes = Shapes(
6     small = RoundedCornerShape(8.dp),
7     medium = RoundedCornerShape(16.dp),
8     large = RoundedCornerShape(16.dp)
9 )

```

Typographie

`ui.theme.Type.kt`

```

1 val Cabin = FontFamily(
2     Font(R.font.cabin, FontWeight.Normal),
3     Font(R.font.cabin_bold, FontWeight.Bold)
4 )
5
6
7 val Typography = Typography(
8     displayLarge = TextStyle(
9         fontFamily = Cabin,
10        fontWeight = FontWeight.Normal,
11        fontSize = 30.sp
12    ),
13    displayMedium = TextStyle(
14        fontFamily = Cabin,
15        fontWeight = FontWeight.Bold,
16        fontSize = 20.sp
17    ),
18    displaySmall = TextStyle(
19        fontFamily = Cabin,
20        fontWeight = FontWeight.Bold,
21        fontSize = 20.sp
22    ),
23    bodyLarge = TextStyle(
24        fontFamily = Cabin,
25        fontWeight = FontWeight.Normal,
26        fontSize = 16.sp
27    )
28 )

```

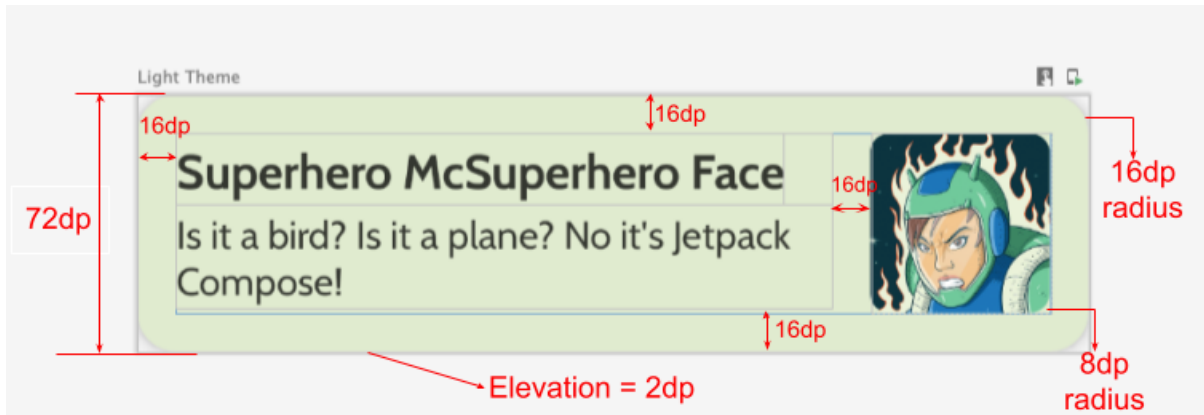
Afficher la liste

La première étape pour concevoir une liste consiste à créer un élément de liste:

1. Dans le fichier `MainActivity.kt`, créer un composant qui représente un élément de liste de super-héros, ressemblant à la capture d'écran et aux spécifications de l'UI suivantes.

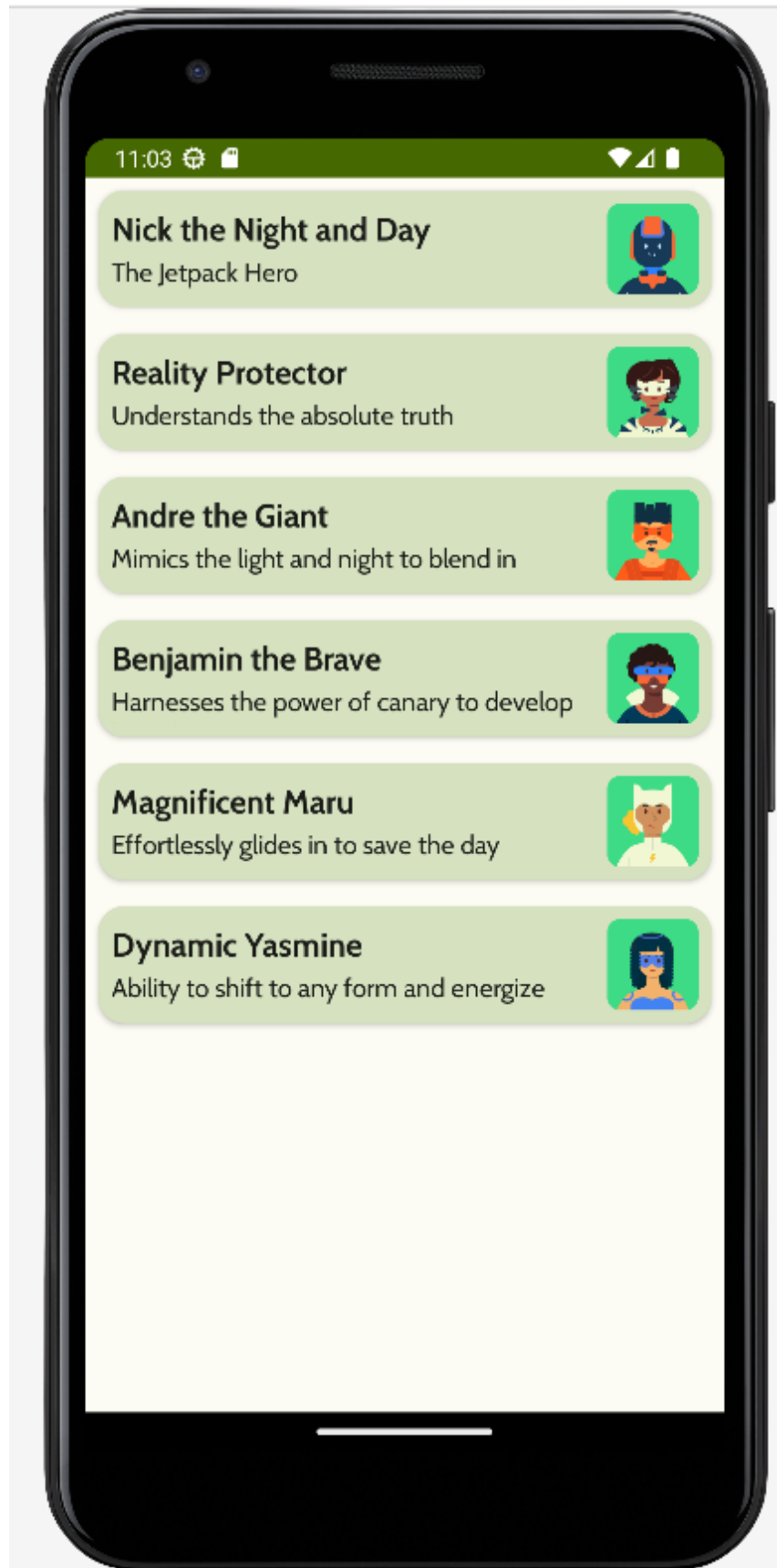
Respectez les spécifications concernant l'interface utilisateur:

- La hauteur de l'élément de liste est de 72 dp.
- La marge intérieure de l'élément de liste est de 16 dp.
- L'élévation de la fiche est de 2 dp.
- L'espace entre l'image et le texte est de 16 dp.
- Le rayon de l'image est de 8 dp.
- Le rayon de l'élément de liste est de 16 dp.
- Le nom de la police du super-héros est `displaySmall`.
- La police de caractères de la description du super-héros correspond à `bodyLarge`.



2. Créez un autre composable qui affiche la liste des héros. C'est ici que vous utiliserez un élément `lazyColumn`.
3. Utilisez les spécifications d'interface utilisateur suivantes pour la marge intérieure (coté 16 dp. haut et bas 8 dp).

Une fois l'implémentation terminée, votre application devrait ressembler à la capture d'écran suivante :



Barre d'application supérieure

Ajoutez une barre d'application supérieure à votre application, qui peut ne contenir que du texte simple.

1. Dans `MainActivity.kt`, ajoutez un composable pour afficher la barre d'application supérieure. Ajoutez du texte à la barre d'application supérieure, tel que le nom de l'application. Centrez-le horizontalement et verticalement.
2. Vous pouvez définir la hauteur de la barre d'application sur 56 dp et la police de caractères sur `displayLarge`.

Utilisez `scaffold` pour afficher la barre d'application supérieure. Pour rendre votre application plus élégante, vous pouvez personnaliser la couleur de la barre d'état en fonction de la couleur de l'arrière-plan

Références

- [Créer une application sur les super-héros](#)